

Class Activity 2 Date: 15.07.2026

1.Smart City Air Quality Monitoring

R Program Code:

```
library(ggplot2)
vaccine <- data.frame(
  District=c("A","B","C","D","E"),
  Rate=c(92,75,68,88,55),
  Age=c("18-30","31-45","46-60","60+","18-30")
)
ggplot(vaccine,aes(District,Rate,fill=Rate))+
  geom_bar(stat="identity")+
  scale_fill_gradient(low="yellow",high="red")
ggplot(vaccine,aes(District,Rate,fill=Age))+
  geom_bar(stat="identity")
```

R Program Output:



Python Program Code:

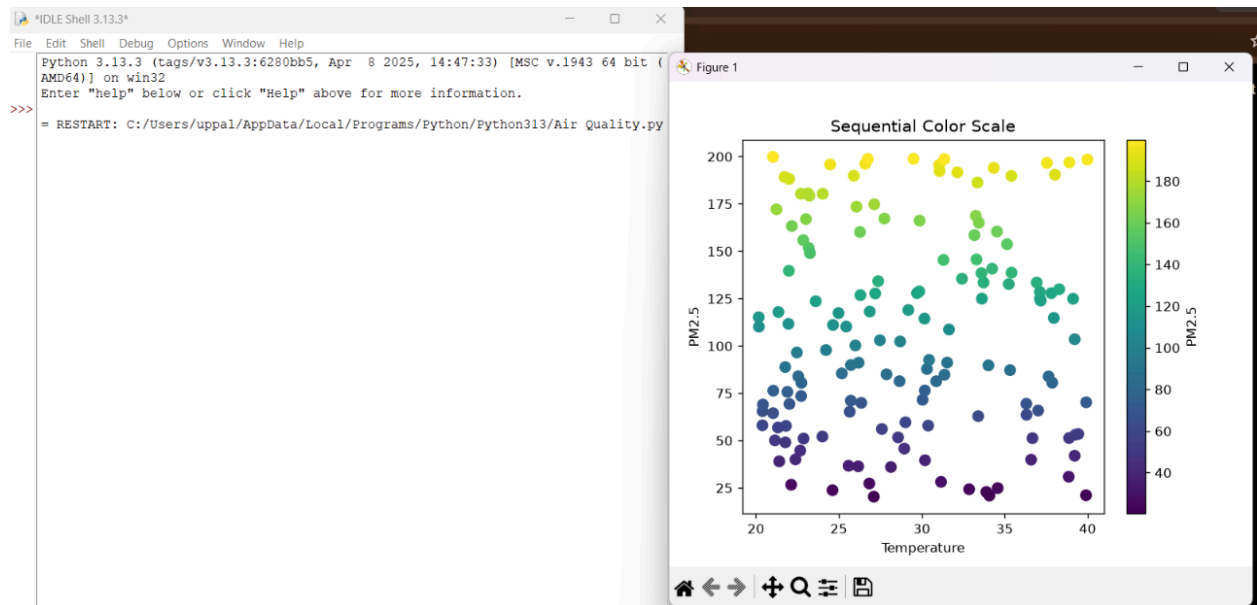
```
import pandas as pd
import matplotlib.pyplot as plt
vaccine=pd.DataFrame({
  "District":["A","B","C","D","E"],
```

```

"Rate":[92,75,68,88,55],
"Age":["18-30","31-45","46-60","60+","18-30"]
})
plt.bar(vaccine["District"],vaccine["Rate"],
color=plt.cm.YlOrRd(vaccine["Rate"]/100))
plt.show()
plt.bar(vaccine["District"],vaccine["Rate"])
plt.show()

```

Python Program Output:



2.Earthquake Risk Analysis

R Program Code:

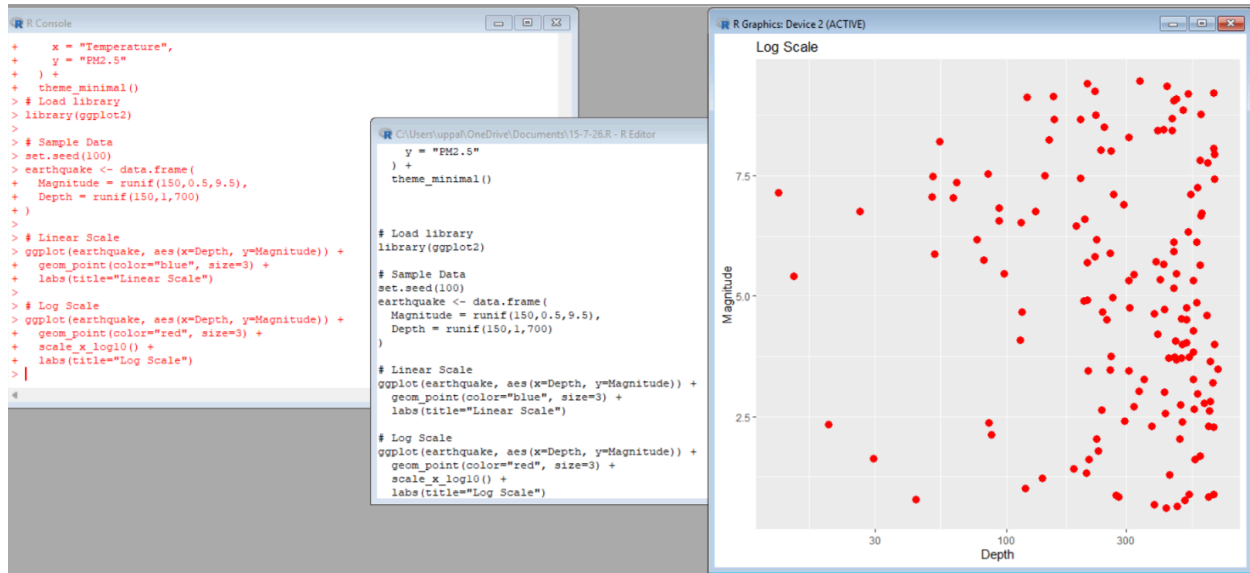
```

library(ggplot2)
set.seed(100)
earthquake <- data.frame(
  Magnitude = runif(150,0.5,9.5),
  Depth = runif(150,1,700)
)
ggplot(earthquake, aes(x=Depth, y=Magnitude)) +
  geom_point(color="blue", size=3) +
  labs(title="Linear Scale")
ggplot(earthquake, aes(x=Depth, y=Magnitude)) +

```

```
geom_point(color="red", size=3) +
scale_x_log10() +
labs(title="Log Scale")
```

R Program Output:

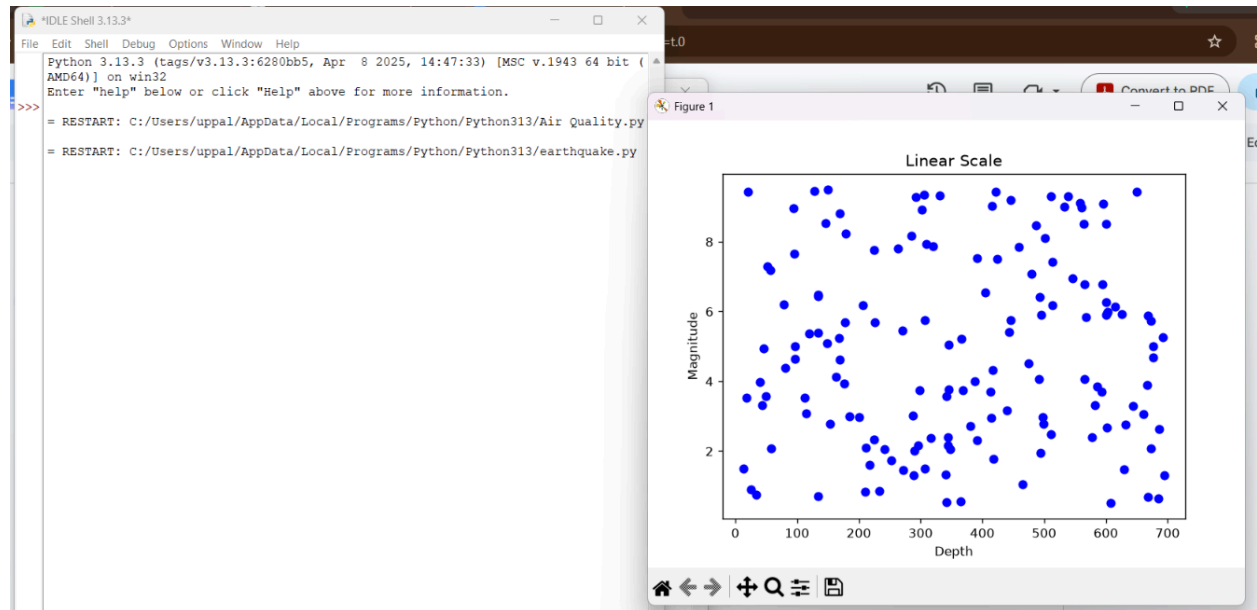


Python Program Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(100)
earthquake = pd.DataFrame({
    "Magnitude": np.random.uniform(0.5,9.5,150),
    "Depth": np.random.uniform(1,700,150)
})
plt.scatter(earthquake["Depth"], earthquake["Magnitude"], color="blue")
plt.title("Linear Scale")
plt.xlabel("Depth")
plt.ylabel("Magnitude")
plt.show()
plt.scatter(earthquake["Depth"], earthquake["Magnitude"], color="red")
plt.xscale("log")
plt.title("Log Scale")
plt.xlabel("Depth (Log Scale)")
plt.ylabel("Magnitude")
```

```
plt.show()
```

Python Program Output:

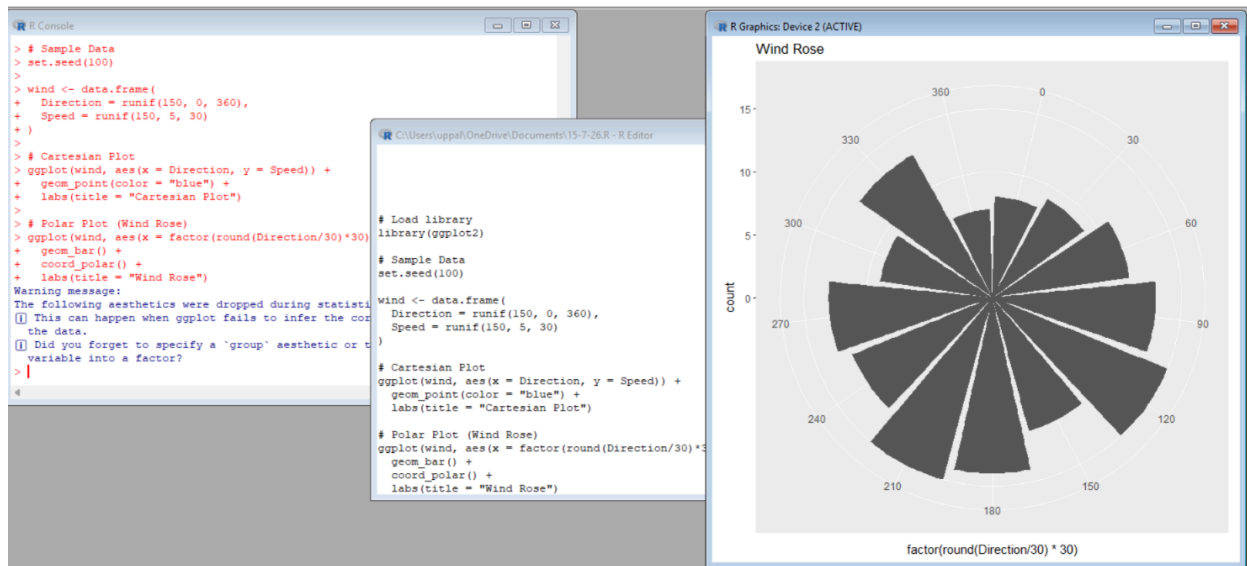


3.Wind Direction Analysis for Airport Operations

R Program Code:

```
library(ggplot2)
set.seed(100)
wind <- data.frame(
  Direction = runif(150, 0, 360),
  Speed = runif(150, 5, 30)
)
ggplot(wind, aes(x = Direction, y = Speed)) +
  geom_point(color = "blue") +
  labs(title = "Cartesian Plot")
ggplot(wind, aes(x = factor(round(Direction/30)*30), fill = Speed)) +
  geom_bar() +
  coord_polar() +
  labs(title = "Wind Rose")
```

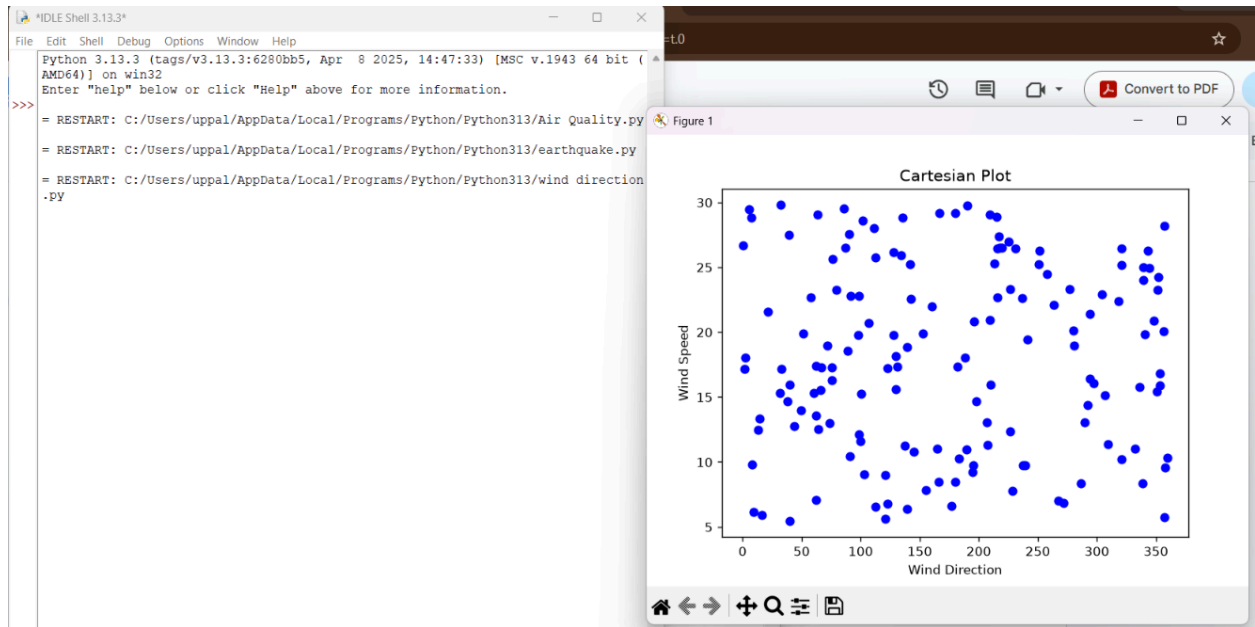
R Program Output:



Python Program Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(100)
wind = pd.DataFrame({
    "Direction": np.random.uniform(0, 360, 150),
    "Speed": np.random.uniform(5, 30, 150)
})
plt.scatter(wind["Direction"], wind["Speed"], color="blue")
plt.title("Cartesian Plot")
plt.xlabel("Wind Direction")
plt.ylabel("Wind Speed")
plt.show()
theta = np.deg2rad(wind["Direction"])
ax = plt.subplot(111, projection="polar")
ax.scatter(theta, wind["Speed"], color="red")
ax.set_title("Wind Rose")
plt.show()
```

Python Program Output:

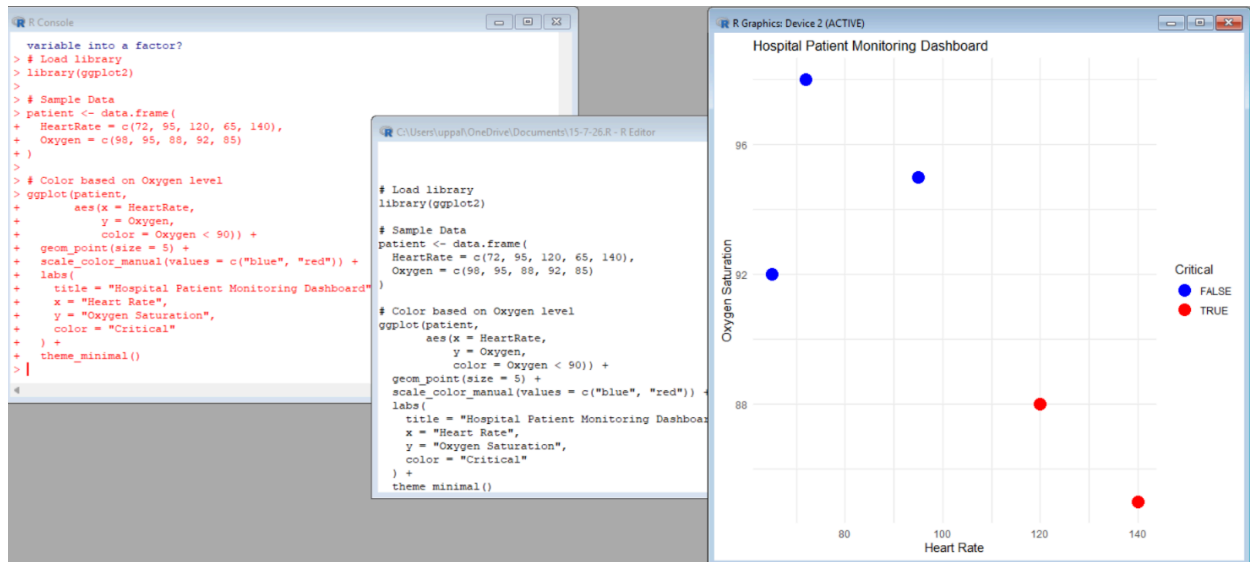


4.Hospital Patient Monitoring Dashboard

R Program Code:

```
library(ggplot2)
patient <- data.frame(
  HeartRate = c(72, 95, 120, 65, 140),
  Oxygen = c(98, 95, 88, 92, 85)
)
ggplot(patient,
  aes(x = HeartRate,
      y = Oxygen,
      color = Oxygen < 90)) +
  geom_point(size = 5) +
  scale_color_manual(values = c("blue", "red")) +
  labs(
    title = "Hospital Patient Monitoring Dashboard",
    x = "Heart Rate",
    y = "Oxygen Saturation",
    color = "Critical"
  ) +
  theme_minimal()
```

R Program Output:



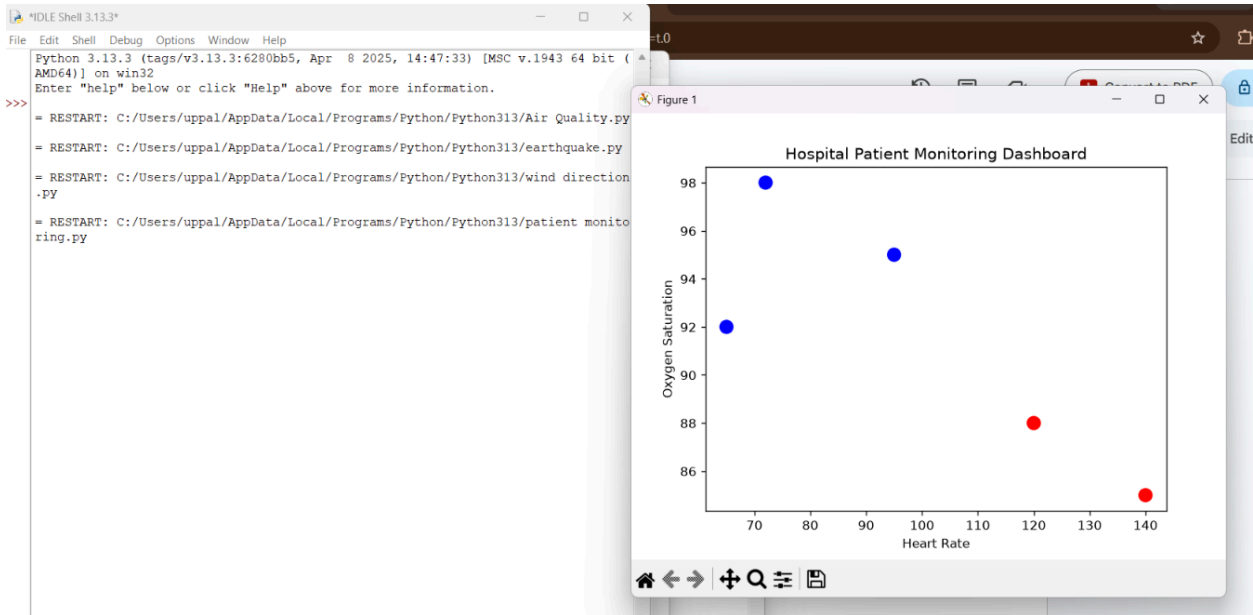
Python Program Code:

```

import pandas as pd
import matplotlib.pyplot as plt
patient = pd.DataFrame({
    "HeartRate": [72, 95, 120, 65, 140],
    "Oxygen": [98, 95, 88, 92, 85]
})
colors = ["red" if x < 90 else "blue" for x in patient["Oxygen"]]
plt.scatter(patient["HeartRate"], patient["Oxygen"],
            c=colors, s=100)
plt.title("Hospital Patient Monitoring Dashboard")
plt.xlabel("Heart Rate")
plt.ylabel("Oxygen Saturation")
plt.show()

```

Python Program Output:

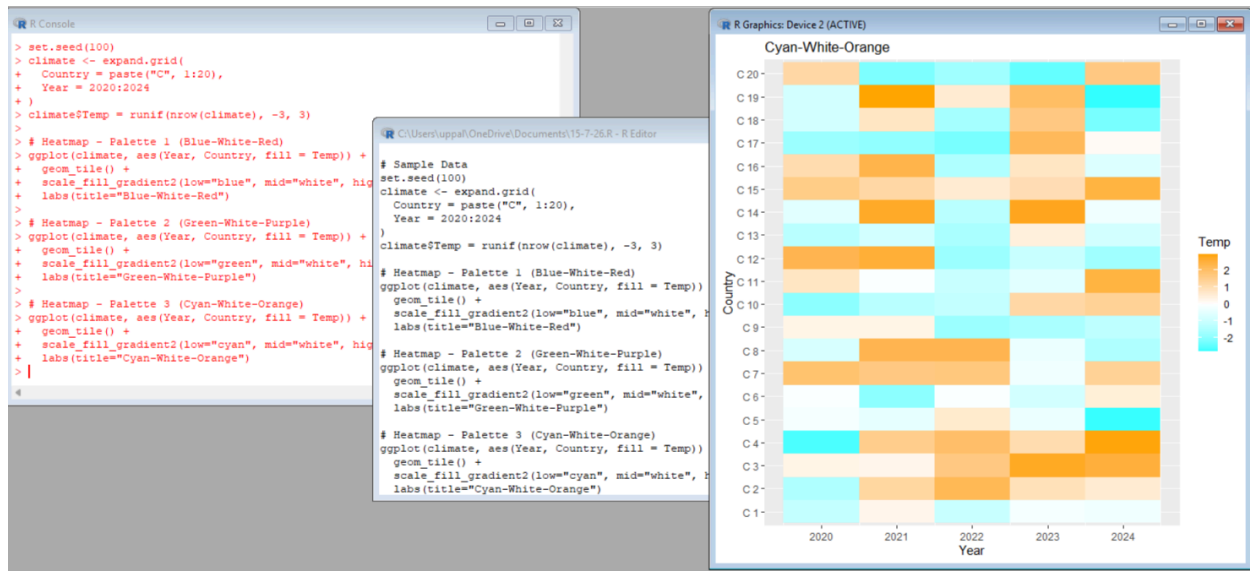


5.Global Climate Change Study

R Program Code:

```
library(ggplot2)
set.seed(100)
climate <- expand.grid(
  Country = paste("C", 1:20),
  Year = 2020:2024
)
climate$Temp = runif(nrow(climate), -3, 3)
ggplot(climate, aes(Year, Country, fill = Temp)) +
  geom_tile() +
  scale_fill_gradient2(low="blue", mid="white", high="red", midpoint=0) +
  labs(title="Blue-White-Red")
ggplot(climate, aes(Year, Country, fill = Temp)) +
  geom_tile() +
  scale_fill_gradient2(low="green", mid="white", high="purple", midpoint=0) +
  labs(title="Green-White-Purple")
ggplot(climate, aes(Year, Country, fill = Temp)) +
  geom_tile() +
  scale_fill_gradient2(low="cyan", mid="white", high="orange", midpoint=0) +
  labs(title="Cyan-White-Orange")
```

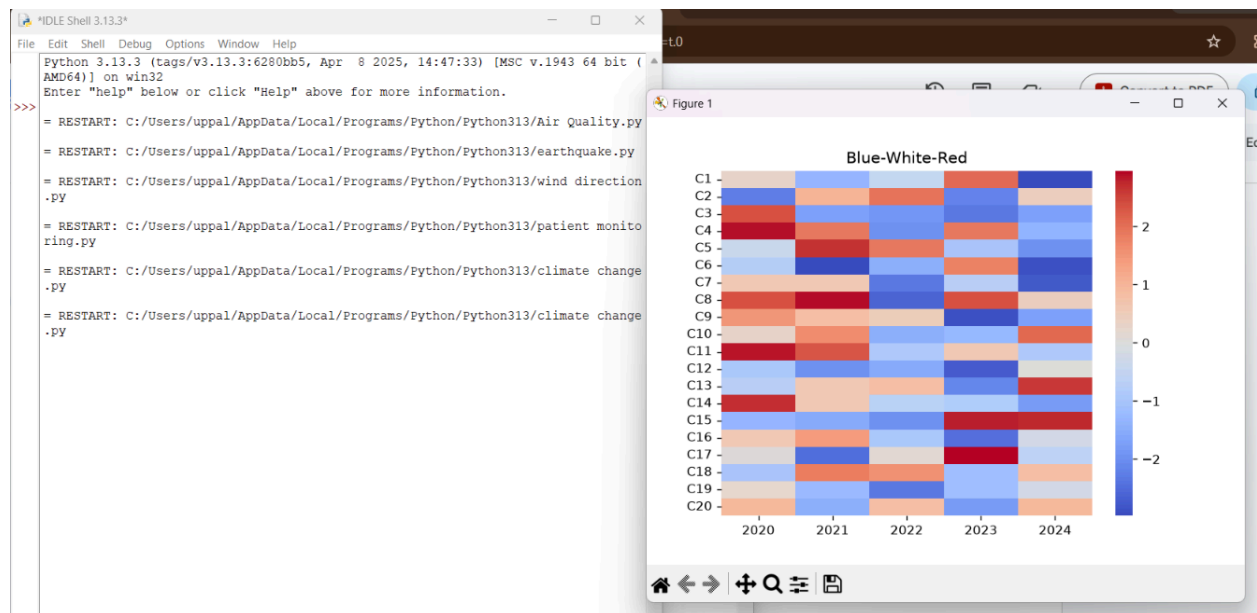

R Program Output:



Python Program Code:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
np.random.seed(100)
data = pd.DataFrame(
    np.random.uniform(-3,3,(20,5)),
    index=[f"C{i}" for i in range(1,21)],
    columns=[2020,2021,2022,2023,2024]
)
sns.heatmap(data, cmap="coolwarm", center=0)
plt.title("Blue-White-Red")
plt.show()
sns.heatmap(data, cmap="PRGn", center=0)
plt.title("Green-White-Purple")
plt.show()
sns.heatmap(data, cmap="PuOr", center=0)
plt.title("Purple-Orange")
plt.show()
```

Python Program Output:

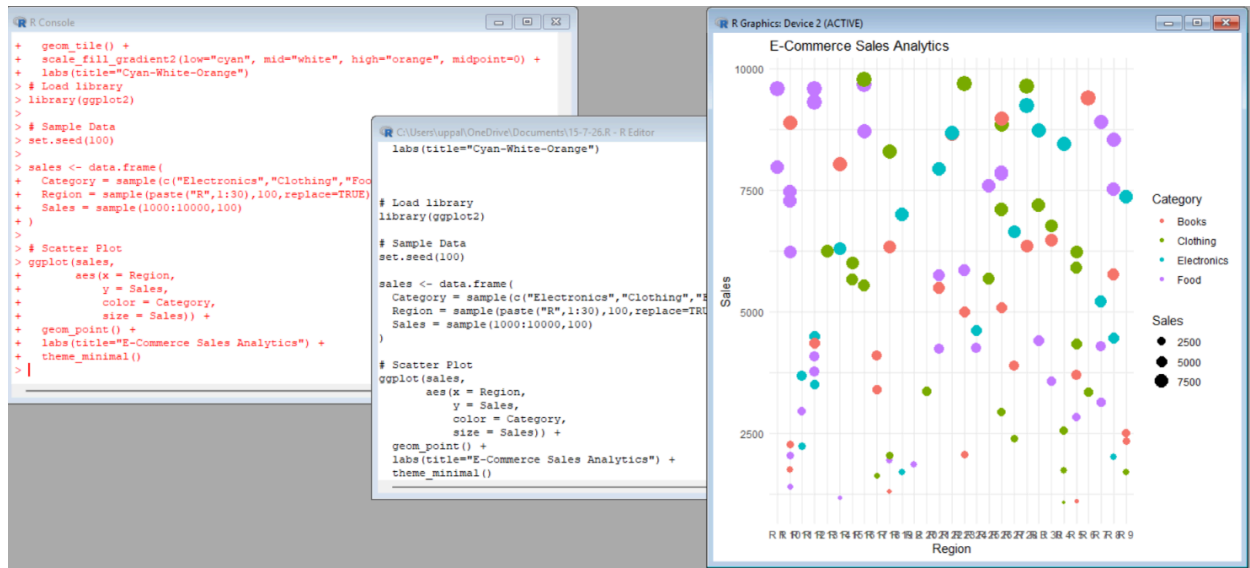


6.E-Commerce Sales Analytic

R Program Code:

```
library(ggplot2)
set.seed(100)
sales <- data.frame(
  Category = sample(c("Electronics","Clothing","Food","Books"),100,replace=TRUE),
  Region = sample(paste("R",1:30),100,replace=TRUE),
  Sales = sample(1000:10000,100)
)
ggplot(sales,
  aes(x = Region,
    y = Sales,
    color = Category,
    size = Sales)) +
  geom_point() +
  labs(title="E-Commerce Sales Analytics") +
  theme_minimal()
```

R Program Output:



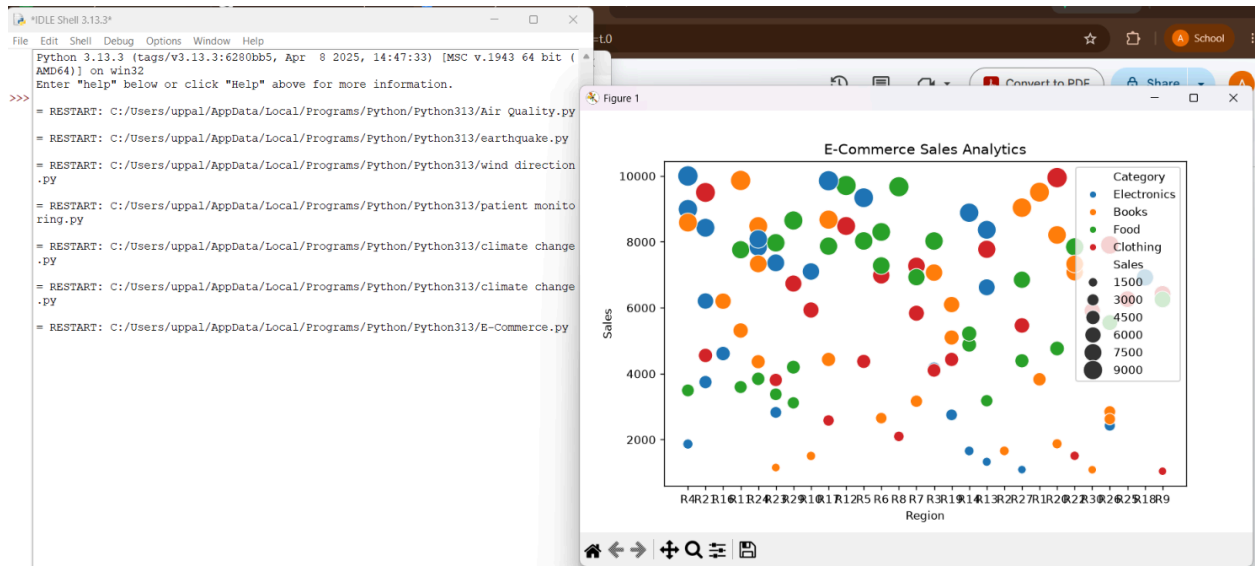
Python Program Code:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
np.random.seed(100)
sales = pd.DataFrame({
    "Category": np.random.choice(["Electronics", "Clothing", "Food", "Books"], 100),
    "Region": np.random.choice([f"R{i}" for i in range(1, 31)], 100),
    "Sales": np.random.randint(1000, 10000, 100)
})
plt.figure(figsize=(8, 5))
sns.scatterplot(data=sales,
                x="Region",
                y="Sales",
                hue="Category",
                size="Sales",
                sizes=(50, 300))
plt.title("E-Commerce Sales Analytics")
plt.show()

```

Python Program Output:

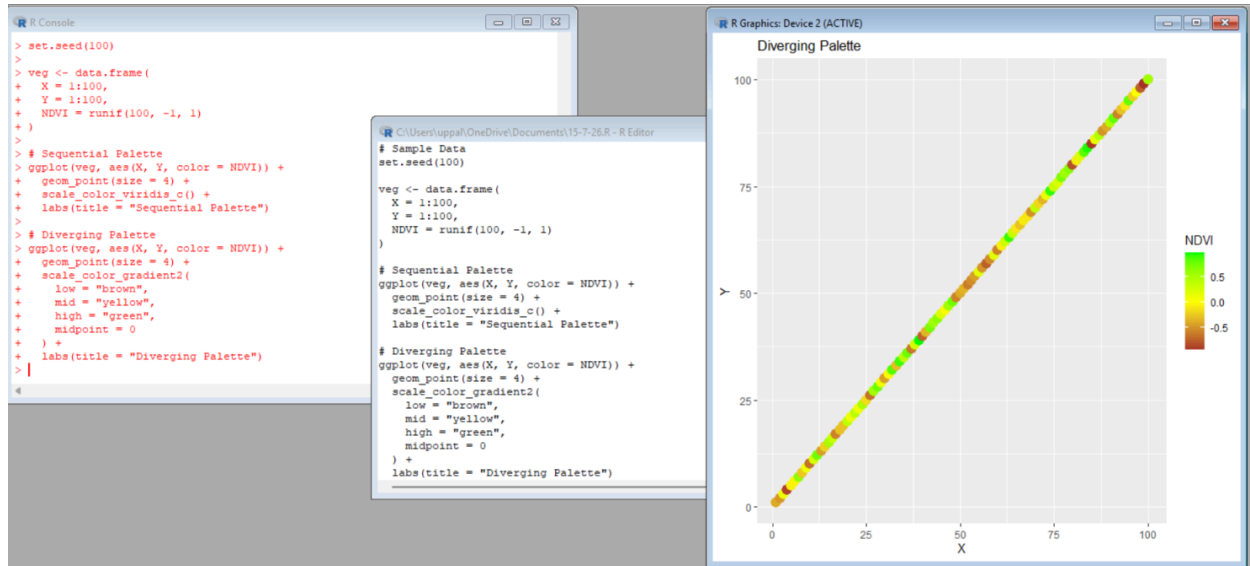


7.Satellite Image Vegetation Analysis

R Program Code:

```
library(ggplot2)
set.seed(100)
veg <- data.frame(
  X = 1:100,
  Y = 1:100,
  NDVI = runif(100, -1, 1)
)
ggplot(veg, aes(X, Y, color = NDVI)) +
  geom_point(size = 4) +
  scale_color_viridis_c() +
  labs(title = "Sequential Palette")
ggplot(veg, aes(X, Y, color = NDVI)) +
  geom_point(size = 4) +
  scale_color_gradient2(
    low = "brown",
    mid = "yellow",
    high = "green",
    midpoint = 0
  ) +
  labs(title = "Diverging Palette")
```

R Program Output:

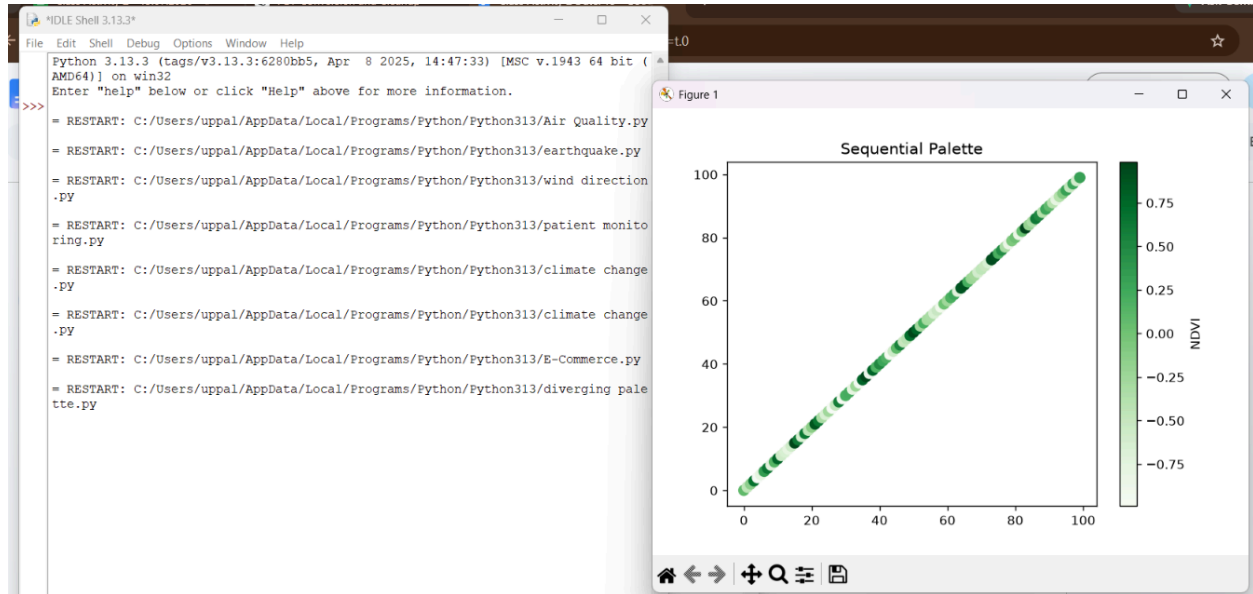


Python Program Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(100)
veg = pd.DataFrame({
    "X": range(100),
    "Y": range(100),
    "NDVI": np.random.uniform(-1,1,100)
})
plt.scatter(veg["X"], veg["Y"],
            c=veg["NDVI"],
            cmap="Greens",
            s=60)
plt.colorbar(label="NDVI")
plt.title("Sequential Palette")
plt.show()
plt.scatter(veg["X"], veg["Y"],
            c=veg["NDVI"],
            cmap="RdYlGn",
            s=60)
plt.colorbar(label="NDVI")
plt.title("Diverging Palette")
```

plt.show()

Python Program Output:



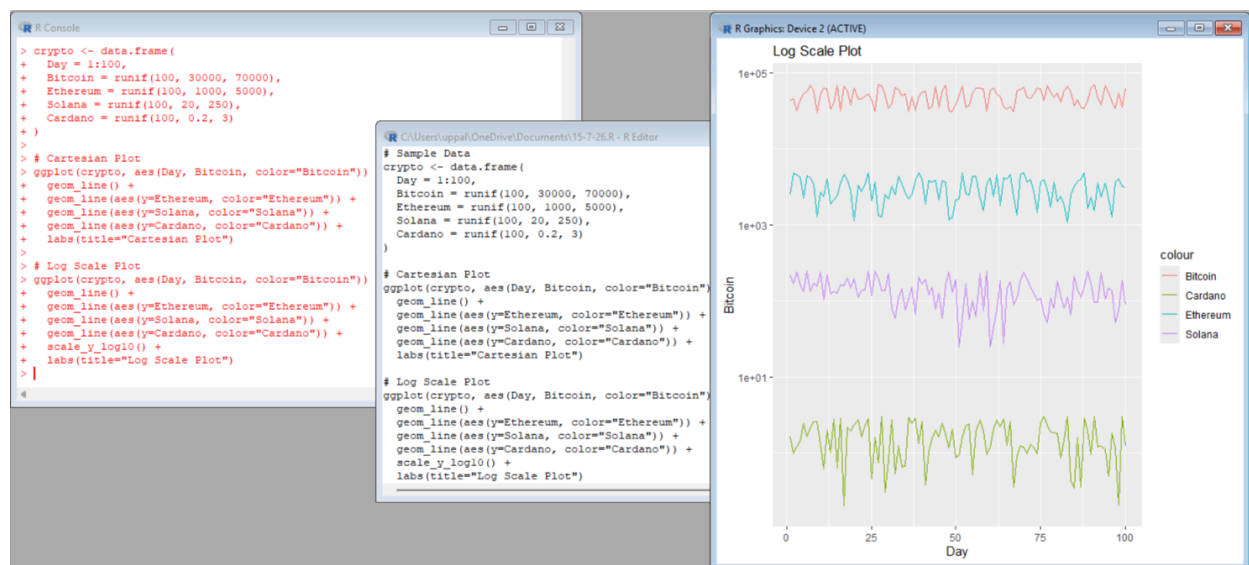
8.Cryptocurrency Market Volatility

R Program Code:

```
library(ggplot2)
crypto <- data.frame(
  Day = 1:100,
  Bitcoin = runif(100, 30000, 70000),
  Ethereum = runif(100, 1000, 5000),
  Solana = runif(100, 20, 250),
  Cardano = runif(100, 0.2, 3)
)
ggplot(crypto, aes(Day, Bitcoin, color="Bitcoin")) +
  geom_line() +
  geom_line(aes(y=Ethereum, color="Ethereum")) +
  geom_line(aes(y=Solana, color="Solana")) +
  geom_line(aes(y=Cardano, color="Cardano")) +
  labs(title="Cartesian Plot")
ggplot(crypto, aes(Day, Bitcoin, color="Bitcoin")) +
  geom_line() +
  geom_line(aes(y=Ethereum, color="Ethereum")) +
```

```
geom_line(aes(y=Solana, color="Solana")) +
geom_line(aes(y=Cardano, color="Cardano")) +
scale_y_log10() +
labs(title="Log Scale Plot")
```

R Program Output:



Python Program Code:

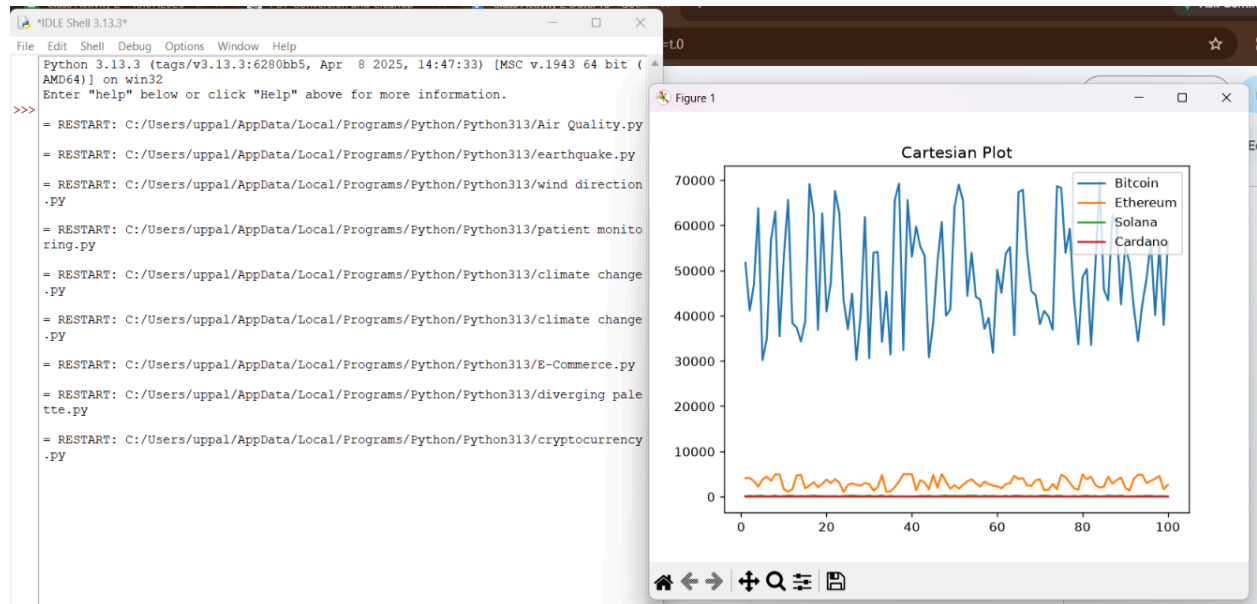
```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(100)
day = np.arange(1,101)
bitcoin = np.random.uniform(30000,70000,100)
ethereum = np.random.uniform(1000,5000,100)
solana = np.random.uniform(20,250,100)
cardano = np.random.uniform(0.2,3,100)
plt.plot(day, bitcoin, label="Bitcoin")
plt.plot(day, ethereum, label="Ethereum")
plt.plot(day, solana, label="Solana")
plt.plot(day, cardano, label="Cardano")
plt.title("Cartesian Plot")
plt.legend()
plt.show()
plt.plot(day, bitcoin, label="Bitcoin")
plt.plot(day, ethereum, label="Ethereum")
```

```

plt.plot(day, solana, label="Solana")
plt.plot(day, cardano, label="Cardano")
plt.yscale("log")
plt.title("Log Scale Plot")
plt.legend()
plt.show()

```

Python Program Output:



9.Marathon Runner Performance Analysis

R Program Code:

```

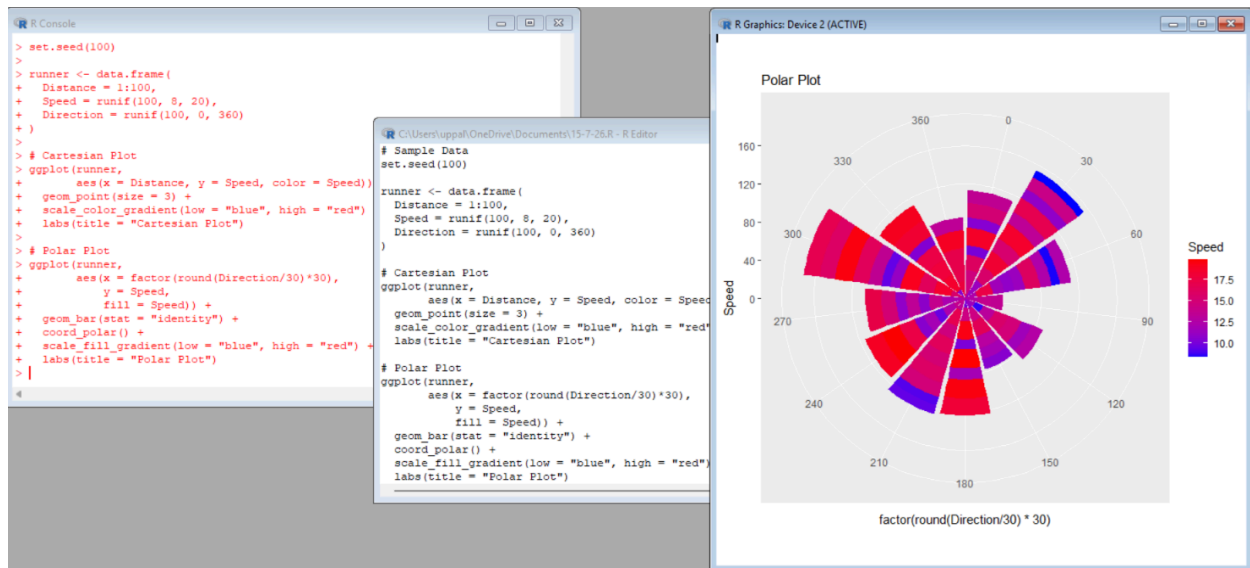
library(ggplot2)
set.seed(100)
runner <- data.frame(
  Distance = 1:100,
  Speed = runif(100, 8, 20),
  Direction = runif(100, 0, 360)
)
ggplot(runner,
  aes(x = Distance, y = Speed, color = Speed)) +
  geom_point(size = 3) +
  scale_color_gradient(low = "blue", high = "red") +
  labs(title = "Cartesian Plot")

```



```
ggplot(runner,
      aes(x = factor(round(Direction/30)*30),
          y = Speed,
          fill = Speed)) +
geom_bar(stat = "identity") +
coord_polar() +
scale_fill_gradient(low = "blue", high = "red") +
labs(title = "Polar Plot")
```

R Program Output:



Python Program Code:

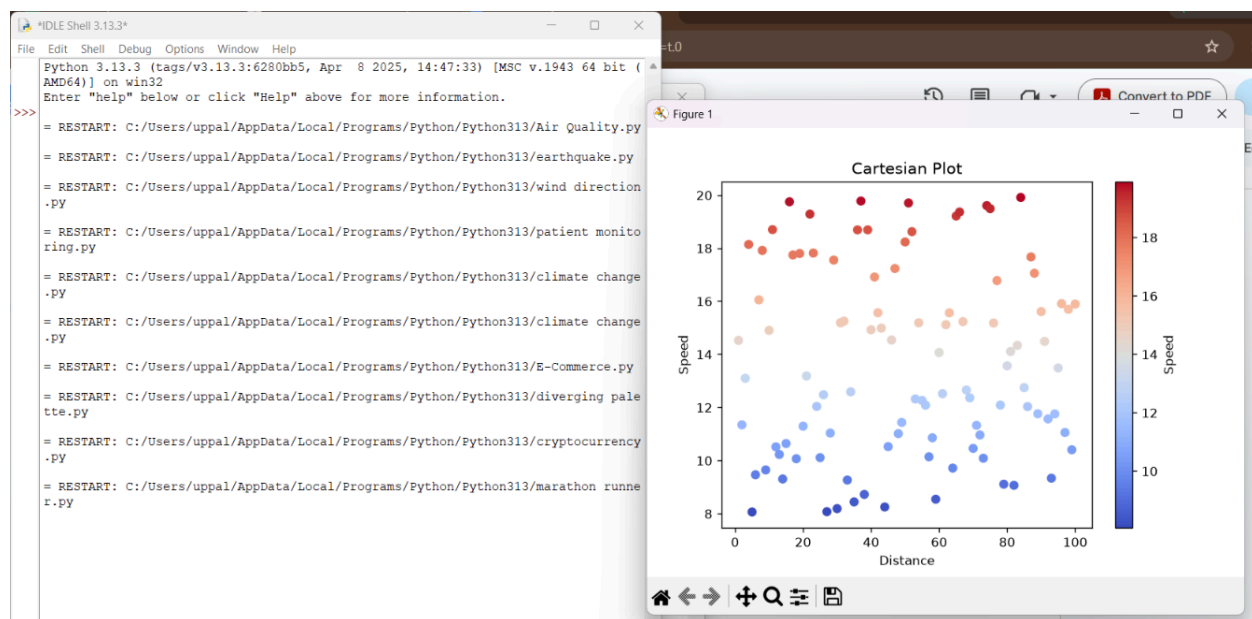
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(100)
runner = pd.DataFrame({
    "Distance": range(1,101),
    "Speed": np.random.uniform(8,20,100),
    "Direction": np.random.uniform(0,360,100)
})
plt.scatter(runner["Distance"],
            runner["Speed"],
            c=runner["Speed"],
            cmap="coolwarm")
```

```

plt.colorbar(label="Speed")
plt.title("Cartesian Plot")
plt.xlabel("Distance")
plt.ylabel("Speed")
plt.show()
theta = np.deg2rad(runner["Direction"])
ax = plt.subplot(111, projection="polar")
ax.scatter(theta,
            runner["Speed"],
            c=runner["Speed"],
            cmap="coolwarm")
plt.title("Polar Plot")
plt.show()

```

Python Program Output:



10.COVID-19 Vaccination Campaign Dashboard

R Program Code:

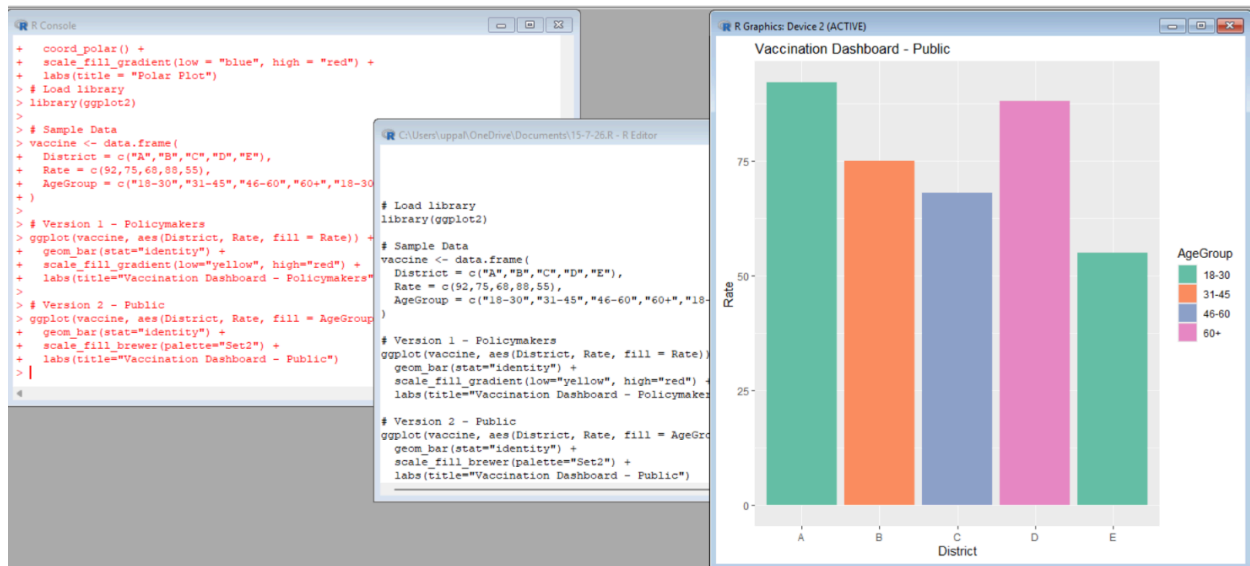
```

library(ggplot2)
vaccine <- data.frame(
  District = c("A","B","C","D","E"),
  Rate = c(92,75,68,88,55),
  AgeGroup = c("18-30","31-45","46-60","60+","18-30")
)

```

```
)
ggplot(vaccine, aes(District, Rate, fill = Rate)) +
  geom_bar(stat="identity") +
  scale_fill_gradient(low="yellow", high="red") +
  labs(title="Vaccination Dashboard - Policymakers")
ggplot(vaccine, aes(District, Rate, fill = AgeGroup)) +
  geom_bar(stat="identity") +
  scale_fill_brewer(palette="Set2") +
  labs(title="Vaccination Dashboard - Public")
```

R Program Output:



Python Program Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
vaccine = pd.DataFrame({
    "District":["A","B","C","D","E"],
    "Rate":[92,75,68,88,55],
    "AgeGroup":["18-30","31-45","46-60","60+","18-30"]
})
plt.bar(vaccine["District"], vaccine["Rate"],
        color=plt.cm.YlOrRd(vaccine["Rate"]/100))
plt.title("Vaccination Dashboard - Policymakers")
```

```

plt.show()
sns.barplot(data=vaccine,
            x="District",
            y="Rate",
            hue="AgeGroup",
            palette="Set2")
plt.title("Vaccination Dashboard - Public")
plt.show()

```

Python Program Output:

